TP4 : AlphaZero pour le jeu de Hex

1 Objectif

L'objectif de ce TP est d'étendre l'agent MCTS développé dans le TP3 en y intégrant un **réseau de neurones convolutif** (CNN), selon l'approche **AlphaZero**. Plutôt que d'évaluer les positions par des simulations aléatoires (rollouts), votre agent s'appuiera sur un réseau de neurones qui fournit simultanément :

- une **distribution de probabilité** sur les coups (politique), guidant l'exploration de l'arbre ;
- une **estimation de la valeur** de la position courante, remplaçant les rollouts aléatoires.

Ce réseau sera entraîné par *self-play* : l'agent joue contre lui-même, collecte des données, puis améliore son réseau itérativement.

2 Principe d'AlphaZero

AlphaZero combine un algorithme MCTS avec un réseau de neurones profond f_θ qui prend en entrée l'état du plateau s et produit deux sorties :

$$f_\theta(s) = (\mathbf{p}, v)$$

où $\mathbf{p}$ est un vecteur de probabilités sur les coups possibles (politique) et $v \in [-1, 1]$ est l'estimation de la valeur de l'état (probabilité de gagner du point de vue du joueur courant).

2.1 Formule PUCT

Dans AlphaZero, la stratégie UCT du TP3 est remplacée par la formule **PUCT** (*Predictor Upper Confidence bounds applied to Trees*). Pour un état S et un coup i, le score PUCT est :

$$\mathrm{PUCT}(S, i) = \underbrace{\frac{S[i].\mathrm{win}}{S[i].\mathrm{nb}}}_{\text{Exploitation}} + \underbrace{C_{\mathrm{puct}} \times p_i \times \frac{\sqrt{S.\mathrm{nb}}}{1 + S[i].\mathrm{nb}}}_{\text{Exploration guidée par le CNN}}$$

où :

- $S[i].\mathrm{win}$ est le nombre de victoires cumulées pour le coup i à partir de S.
- $S[i].\mathrm{nb}$ est le nombre de simulations passées par le coup i à partir de S.
- $S.\mathrm{nb}$ est le nombre total de simulations à partir de S.
- p_i est la probabilité *a priori* du coup i fournie par le CNN (la composante correspondante du vecteur $\mathbf{p}$).
- C_{puct} est une constante d'exploration (par défaut : 2.5).

Convention : Lorsqu'un coup i n'a jamais été visité ($S[i].\mathrm{nb} = 0$), le terme d'exploitation vaut 0. Le score se réduit alors à :

$$\mathrm{PUCT}(S, i) = C_{\mathrm{puct}} \times p_i \times \sqrt{S.\mathrm{nb}}$$

La différence fondamentale par rapport à UCT est que le terme d'exploration est **pondéré par la probabilité** p_i issue du réseau de neurones. Ainsi, les coups que le CNN juge prometteurs sont explorés en priorité, ce qui rend la recherche beaucoup plus efficace.

2.2 Remplacement des rollouts par la Value Head

Dans le MCTS classique (TP3), lorsqu'un nœud feuille est atteint, une simulation aléatoire (*rollout*) est effectuée pour estimer la valeur de la position. Dans AlphaZero, cette simulation est **remplacée** par l'estimation v du réseau de neurones. Cela présente deux avantages majeurs :

- L'évaluation est beaucoup plus rapide qu'un rollout complet.
- L'estimation est, après entraînement, bien plus précise qu'une simulation aléatoire.

Concrètement, lors de l'expansion d'un nœud feuille, au lieu de jouer aléatoirement jusqu'à la fin de la partie, on appelle la fonction `evaluateState` qui retourne à la fois la distribution de politique **p** (pour initialiser les probabilités *a priori* des coups enfants) et la valeur v (qui est directement rétropropagée dans l'arbre).

Remarque : L'option `--unactivate-value-head` permet de désactiver la tête de valeur du CNN. Dans ce cas, la valeur v retournée par le réseau est ignorée et des rollouts aléatoires sont utilisés à la place, comme dans le TP3. Cela peut être utile pour tester l'effet de la politique seule, ou au début de l'entraînement lorsque la tête de valeur n'est pas encore fiable.

3 Code fourni

3.1 `HexCNN.h` — Le réseau de neurones

Ce fichier contient tout le code du CNN. Il est fourni et **ne nécessite aucune modification**. Les fonctions principales sont :

- `encodeBoardState(cells, size, currentPlayer)` — Encode le plateau en un tenseur `[1, 2, N, N]` pour le CNN.
- `encodePolicy(visit_counts, size, currentPlayer, totalVisits)` — Encode les visites MCTS en une distribution de probabilité (tenseur `[N*N]`) pour l'apprentissage.
- `evaluateState(net, cells, size, currentPlayer)` — Fonction centrale pour le MCTS. Retourne une paire : un `std::vector<float>` de taille $N \times N$ (politique, avec probabilité 0 pour les cases occupées) et un `float` $v \in [-1, 1]$ (valeur estimée).
- `TrainingExample` — Structure contenant un exemple d'entraînement (`state`, `policy`, `value_target`, `player`). Les champs `state` et `policy` se renseignent via `encodeBoardState` et `encodePolicy`.

3.2 `main.cpp` — Le programme principal

L'option `--help` affiche l'aide. Deux modes de fonctionnement :

- **Mode jeu** : identique au TP3, avec le joueur `IANN` disponible via `-X IANN` ou `-O IANN`.
- **Mode entraînement** (`--train`) : exécute la boucle d'apprentissage par *self-play*.

4 Travail à réaliser

Vous devez compléter la classe `IANN_Player` dans le fichier `src/IANN_Player.h`. Vous pouvez réutiliser en grande partie le code de votre MCTS du TP3 (`IA_Player`). Les différences principales sont décrites ci-dessous.

4.1 Méthode `getMove(Hex_Environement& hex)`

C'est la méthode principale qui détermine le coup à jouer. Vous devez implémenter un MCTS utilisant la formule PUCT décrite en section 2.1. Les modifications par rapport au TP3 sont :

1. **Sélection** : Utilisez la formule PUCT au lieu d'UCT. Chaque nœud de l'arbre doit stocker les probabilités *a priori* p_i issues du CNN.
2. **Expansion** : Lorsque vous créez un nouveau nœud, appelez `evaluateState` pour obtenir la distribution de politique $\mathbf{p}$ et la valeur v. Stockez $\mathbf{p}$ dans le nœud (les p_i des coups enfants).
3. **Simulation** : Si la *Value Head* est activée (comportement par défaut), la valeur v retournée par `evaluateState` remplace le rollout. Sinon (si `unactivateValueHead()` a été appelée), effectuez un rollout aléatoire comme dans le TP3.
4. **Rétropropagation** : Identique au TP3, mais en utilisant v au lieu du résultat d'un rollout.

4.2 Méthode `unactivateValueHead()`

Cette méthode désactive l'utilisation de la tête de valeur du CNN. Lorsqu'elle est appelée, votre MCTS doit utiliser des rollouts aléatoires pour estimer la valeur d'un nœud feuille (comme dans le TP3), tout en conservant l'utilisation de la politique du CNN pour guider l'exploration via PUCT.

4.3 Méthode `enableDataCollection(std::vector<TrainingExample>* storage)`

Cette méthode active le **mode entraînement**. À chaque coup joué, votre agent doit sauvegarder un `TrainingExample` dans le vecteur pointé, en renseignant `state` (via `encodeBoardState`), `policy` (via `encodePolicy`), et `player` (`'X'` ou `'O'`). Le champ `value_target` doit être laissé à 0 : il sera rempli automatiquement par `main.cpp` à la fin de la partie.

De plus, lorsque le mode entraînement est actif, le comportement de votre agent doit différer du mode compétitif sur deux points importants :

4.3.1 Sélection du coup par tirage proportionnel

En mode compétitif, le coup final est choisi de manière **déterministe** : on joue le coup le plus visité à la racine. En phase d'apprentissage, on souhaite au contraire encourager l'**exploration** afin de diversifier les positions rencontrées et d'enrichir les données d'entraînement.

Pour cela, le coup est choisi par **tirage au sort proportionnel aux visites**. La probabilité de sélectionner le coup i est :

$$\Pr(\text{coup } i) = \frac{N(\text{racine}, i)}{\sum_j N(\text{racine}, j)}$$

4.3.2 Bruit de Dirichlet à la racine

En mode entraînement, afin d'encourager l'exploration et d'éviter que le réseau ne converge vers une politique trop déterministe, un **bruit de Dirichlet** est ajouté aux probabilités *a priori* p_i du nœud racine. La formule est :

$$p'_i = (1 - \varepsilon) \times p_i + \varepsilon \times \eta_i$$

où :

- $\varepsilon = 0.25$ est le poids du bruit (valeur classique d'AlphaZero).
- $\eta \sim \text{Dir}(\alpha)$ est un vecteur tiré d'une distribution de Dirichlet, avec $\alpha = 0.1 \times N^2$. La méthode `IANN_Player::get_dirichlet_noise` présente dans le code fourni permet de générer un tableau avec un bruit de dirichlet.

Ce bruit n'est ajouté **qu'au nœud racine**.

4.4 Méthode `MCTS_TimeLimit(unsigned int time_limit_ms)`

Cette méthode fixe le temps maximal (en millisecondes) alloué pour chaque coup de l'IA. Assurez-vous que votre MCTS respecte cette limite de temps.
Attention : Vous devez également implémenter cette méthode dans votre classe `IA_Player` du TP3, si ce n'est pas déjà fait, afin qu'elle soit compatible avec le nouveau programme principal.

5 Améliorations possibles (Bonus)

Une fois votre implémentation de base fonctionnelle, vous pouvez chercher à **améliorer la vitesse de convergence** de l'entraînement. Une piste particulièrement efficace consiste à enrichir la représentation de l'état du plateau en ajoutant des **canaux (channels) supplémentaires** en entrée du CNN.

Par défaut, l'encodage du plateau utilise 2 canaux (un par joueur). Or, le réseau doit *apprendre* à partir de ces seules informations brutes toutes les propriétés structurelles du jeu de Hex, ce qui peut être lent. En fournissant des **features pré-calculées** sous forme de canaux additionnels, vous aidez le réseau à converger plus rapidement en lui donnant accès à des informations stratégiques qu'il n'a pas besoin de redécouvrir par lui-même.

Exemples de features intéressantes :

- **Connectivité garantie aux bords** : Pour chaque joueur, un canal binaire où l'on place un 1 sur chaque case occupée par ce joueur qui est **garantie** de pouvoir être connectée à un bord (c'est-à-dire qu'il existe un chemin de cases occupées ou vides menant à un bord du joueur).
- **Ponts virtuels (virtual connections)** : Un canal marquant les cases qui forment des ponts virtuels, c'est-à-dire des paires de cases vides qui garantissent une connexion entre deux groupes, quel que soit le coup de l'adversaire.

Pour ajouter des canaux, vous devrez modifier la fonction `encodeBoardState` dans `HexCNN.h` afin que le tenseur d'entrée passe de la forme `[1, 2, N, N]` à `[1, 2+K, N, N]` (où K est le nombre de canaux ajoutés), et adapter en conséquence la première couche de convolution du réseau.

6 Rendu

Le rendu attendu est une **archive ZIP** contenant les éléments suivants :

- Le **code source** complet de votre projet. L'archive ne doit **pas** contenir de fichiers binaires compilés (pas de dossier `build/`, pas d'exécutable). Le projet doit être compilable avec **CMake** de manière standard (`mkdir build && cd build && cmake .. && make`).
- Les deux modèles entraînés :
  - `hex_cnn_4.pt` — modèle entraîné pour le plateau 4×4.
  - `hex_cnn_8.pt` — modèle entraîné pour le plateau 8×8.

Attention : L'entraînement du modèle 8×8 est significativement plus long que celui du modèle 4×4. Prévoyez **au moins 12 heures d'entraînement** pour le modèle `hex_cnn_8.pt`. Il est donc fortement recommandé de lancer cet entraînement suffisamment tôt et de ne pas attendre la dernière minute.